Infraestrutura de Hardware

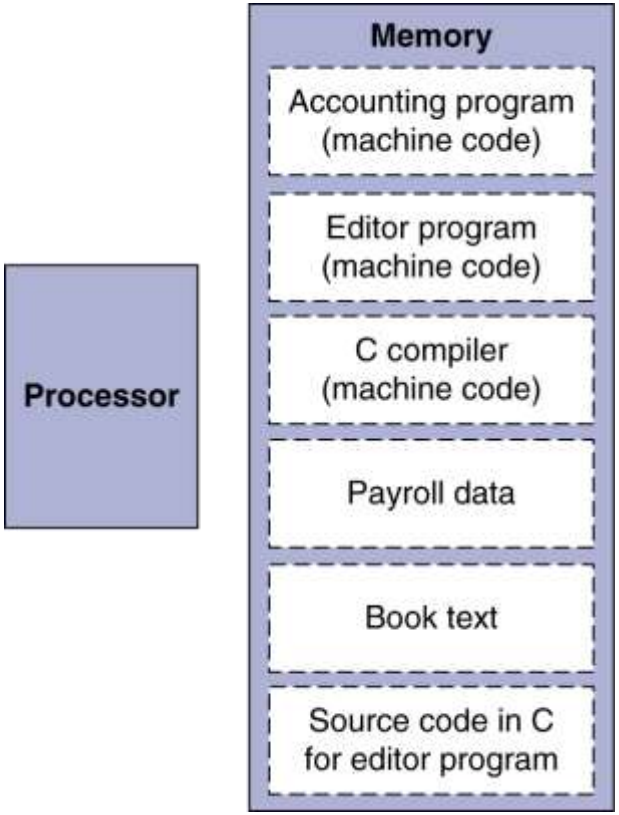
Instruindo um processador real

Prof. Adriano Sarmento

Centro de Informática UFPE

Universidade Federal de Pernambuco

Computadores de Programas Armazenados

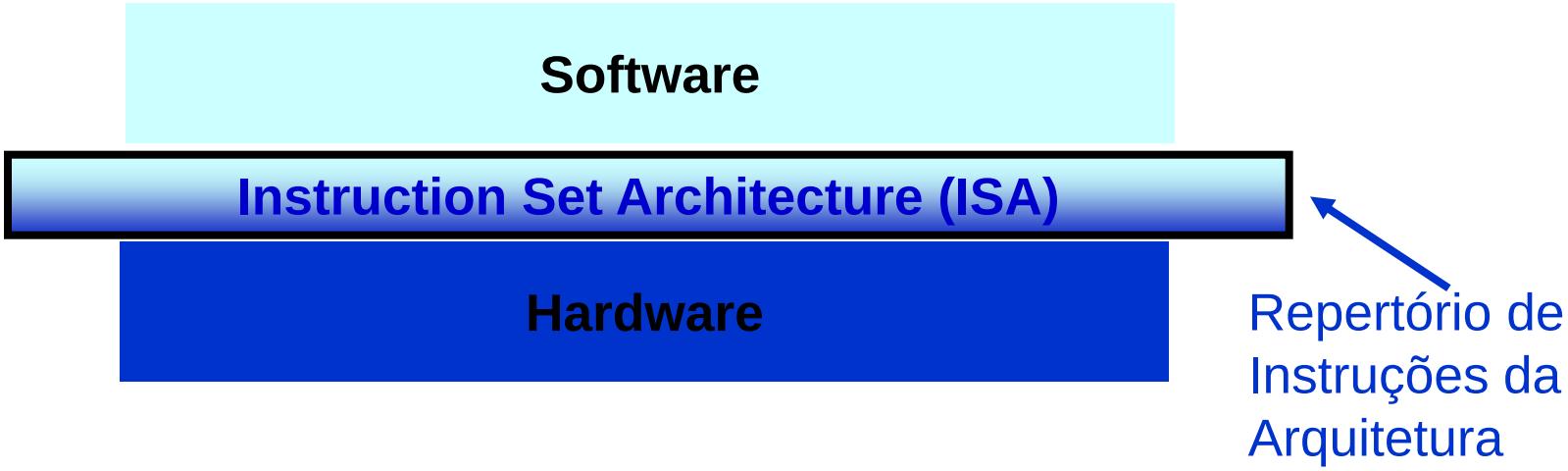


- Instruções e dados representados como números binários
- Instruções e dados armazenados em memória
- Programas podem interagir com outros programas
  - Ex: compiladores, linkers, …
- Compatibilidade de formatos binários permite que programas compilados funcionem em diferentes computadores
  - ISAs padronizadas

Perguntas que Devem ser Respondidas ao Final do Curso

- Como um programa escrito em uma linguagem de alto nível é entendido e executado pelo HW?
- **Qual é a interface entre SW e HW e como o SW instrui o HW a executar o que foi planejado?**
- O que determina o desempenho de um programa e como ele pode ser melhorado?
- Que técnicas um projetista de HW pode utilizar para melhorar o desempenho?

Interface HW/SW: Repertório de Instruções da Arquitetura



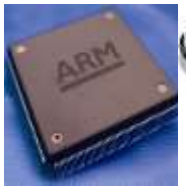
- Última abstração do HW vista pelo SW
- Computadores diferentes podem ter diferentes ISAs
  - Mas com muitos aspectos em comum
- Visando portabilidade de código, indústria se alinha em torno de quantidade pequena de ISAs diferentes

Evolução de ISAs

- Até metade da década de 60 computadores tinham ISAs com quantidade reduzida de instruções e instruções simples
  - Simplifica implementação
- Fim da década de 60 surge ISAs com grande número de instruções complexas
  - **Complex Instruction Set Computer (CISC)**
  - Difícil implementação e existência de muitas instruções pouco usadas
- Começo da década de 80 ISAs com instruções simples voltam a ser comuns
  - **Reduced Instruction Set Computer (RISC)**

Exemplos de Processadores CISC e RISC

- CISC
  - Intel x86, Pentium, AMDx86, AMD Athlon
  - Muito utilzados em PCs
- RISC
  - MIPS, SPARC, ARM, PowerPC
  - Muito utilizados em sistemas embarcados



- Tendência hoje é termos processadores híbridos
  - Ideias de RISC foram incorporados a CISC e vice-versa

Repertório (ISA) do Processador MIPS

- Utilizado como exemplo nesta disciplina
- Desenvolvido no começo de 80, é um bom exemplo de uma arquitetura RISC
- Muito utilizado no mercado de sistemas embarcados
  - Aplicações em eletrônicos diversos, equipamento de rede/armazenamento, câmeras, impressoras, blue-rays, smart watches,etc

Princípios de Projeto do MIPS (RISC)

- Simplicidade é favorecida pela regularidade
  - Instruções de tamanho fixo
  - Poucos formatos de instruções
  - Opcode sempre utiliza os primeiros 6 bits
- Quanto menor, mais rápido
  - Repertório de instruções limitados
  - Quantidade de registradores limitados
  - Número reduzido de modos de endereçamento
- Torne rápido o caso mais comum
  - Existência de instruções que contém operandos
- Bom projeto requer boas escolhas (compromissos)
  - Diferentes formatos de instruções complica decodificação, CONTUDO permite instruções de tamanho fixo

Categoria e Formato de Instruções do MIPS

- Aritméticas
- Lógicas
- Transferência de dados
- Desvios de fluxo
- Gerenciamento de memória

3 Formatos de Instrução: todos com 32 bits

op	**rs**	**rt**	**rd**	**shamt**	**funct**	R format
op	**rs**	**rt**	**immediate**			I format
op	**jump target**					J format

Bom projeto requer boas escolhas (compromissos)

Operações Aritméticas

- Maioria das instruções aritméticas no MIPS possuem 3 operandos:
  - destino, fonte 1, fonte 2

- Cada instrução aritmética faz apenas uma operação

```
add a,b,c
```

→

```
a = b + c
```

```
sub a,b,c
```

→

```
a = b - c
```

A simplicidade é favorecida pela regularidade

Exemplo: Expressões Aritméticas

Código C

```
f = (g + h) -(i + j);
```

Pseudo-código Assembly MIPS

```
add t0,g,h #temp t0 = g + h
add t1,i,j #temp t1 = i + j
sub f,t0,t1 # f = t0 - t1
```

Operandos nos Registradores

- Para melhorar desempenho, os operandos de uma instrução aritmética devem estar nos registradores
  - Acesso mais rápido em relação à memória
- Todos os registradores no MIPS possuem 32 bits
  - 32 bits é uma **palavra** (word) no MIPS
- Número de registradores no MIPS é reduzido: 32
  - Número grande de registradores pode penalizar desempenho
  - Impacto no tamanho da instrução

Quanto menor, mais rápido

Registradores para Operandos no MIPS

- Nomes assembly dos registradores
  - **$s0, $s1…$s7 – armazenam variáveis dos programas**
  - **$t0, $t1…$t9 – para valores temporários**

Código C

```
f = (g + h) -(i + j);
```

Código Assembly MIPS

```
add $t0,$s1,$s2 # $s1 = g , $s2 = h
add $t1,$s3,$s4 # $s3 = i ,$s4 = j
sub $s0,$t0,$t1 # $s0 = f = t0 - t1
```

Representação das instruções

- Informação tem uma representação numérica na base 2
  - Codificação das instruções
  - Mapeamento de nomes de registradores para números
    - $s0 a $s7 : 16 a 23
    - $t0 a $t7 : 8 a 15
    - $t8-$t9 : 24-25

Formato da Instrução `ADD` e `SUB`

Formato R de Instrução

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

op	opcode da instrução
rs	registrador que contém 1° operando fonte (source)
rt	registrador que contém o 2° operando fonte
rd	registrador destino que contém resultado
shamt	shift amount , não utilizado para **`add`** e **`sub`**
funct	função (function) que estende o opcode

Representando ADD na Máquina

- Número dos registradores
  - **$s0 - $s7: 16 - 23**
  - **$t0 - $t7 : 8-15**
  - **$t8-$t9 : 24-25**

Código Assembly MIPS

```
add $t0,$s1,$s2
```

	$s1	$s2	$t0		
0	17	18	8	0	32
op	rs	rt	rd	shamt	funct
000000	10001	10010	01000	00000	100000

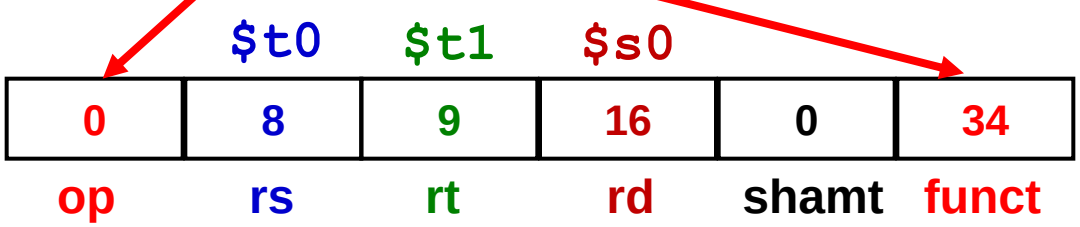
Em binário

Representando SUB na Máquina

- Número dos registradores
  - **$s0 - $s7: 16 - 23**
  - **$t0 - $t7 : 8-15**
  - **$t8-$t9 : 24-25**

Código Assembly MIPS

```
sub $s0,$t0,$t1
```



Operações de Transferência de Dados (Memória)

- Operações de transferência de dados entre processador e memória no MIPS tem dois operandos:
  - registrador, endereço de memória
  - Endereço de memória formado por um registrador de base (contém endereço inicial) e um deslocamento(offset)

- Duas operações básicas: lw (load word), sw (store word)

```
lw r, offset(end_inicial)
```

➡ **Carrega conteúdo de `offset` + `end_inicial` no registrador `r`**

```
sw r, offset(end_inicial)
```

➡ **Armazena conteúdo de registrador `r` em `offset` + `end_inicial`**

Operandos na Memória

- Memória muito utilizada para armazenar dados compostos
  - Arrays, estruturas, dados dinâmicos
- Memória endereçada por byte (8 bits)
- Contudo memória é vista como uma sequência de palavras de 32 bits
  - **Endereços de palavras devem ser múltiplos de 4!**
- MIPS é Big Endian
  - Byte mais significativo tem menor endereço da palavra
  - Little Endian – byte mais significativo tem o maior endereço

Endereços da Memória – Big Endian

- MIPS é Big Endian

0	**0x12**
1	**0x34**
2	**0x68**
3	**0x00**

$s0 = 0

lw $t0, 0($s0)

$t0 = 0x12346800

Endereços da Memória

- MIPS
  - Inteiros de 32 bits (4 bytes)
  - Array de inteiros chamado de a

End (a[0]) = 20
End (a[1]) = 24

End(a[i]) = End-inicial + i x 4

Utilizando Arrays no MIPS (Memória)

- Registrador de base guarda endereço inicial do array
  - Offset utilizado como índice do array

Código C

```
a[12] = h + a[8];
```

`h` em `$s2`, endereço base de `a` em `$s3`

Código Assembly MIPS

```
lw $t0,32($s3)
add $t0,$s2,$t0
sw $t0,48($s3)
```

Índice 8 requer offset de 32 (4 x 8)

Outro Exemplo de Arrays no MIPS

- Array com variável de indexação

Código C

```
g = g + a[i];
```

g em $s1, i em $s2, endereço base de a em $s3

Código Assembly MIPS

```
add $t1, $s2, $s2
add $t1, $t1, $t1
add $t1, $t1, $s3
lw  $t0, 0($t1)
add $s1, $s1, $t0
```

End(a[i]) = base + i + i + i + i

Registradores x Memória

- Acesso a registradores é mais rápido
- Utilização da memória requer *loads* e *stores*
  - Mais instruções a serem executadas
- Compilador deve maximizar a utilização de registradores
  - **Otimização de registradores é importante!**

Formato da Instrução `LW` e `SW`

Formato I de Instrução

op	rs	rt	constant or address
6 bits	5 bits	5 bits	16 bits

op	opcode da instrução
rs	registrador que neste caso contém endereço base
rt	registrador fonte ou destino
constant	constante que representa o offset

Representando `LW` na Máquina

- Número dos registradores
  - **$s0 - $s7: 16 - 23**
  - **$t0 - $t7 : 8-15**
  - **$t8-$t9 : 24-25**

Código Assembly MIPS

```
lw $t0,32($s3)
```

	$s3	$t0	
35	19	8	32
op	rs	rt	constant

Representando sw na Máquina

- Número dos registradores
  - **$s0 - $s7: 16 - 23**
  - **$t0 - $t7 : 8-15**
  - **$t8-$t9 : 24-25**

Código Assembly MIPS

```
sw $t0,48($s1)
```

	$s1	$t0	
43	17	8	48
op	rs	rt	constant

Operandos Imediatos

- Frequentemente utilizamos pequenas constantes em programas
  - Ex: `a = a + 1;`
- Possíveis soluções
  - Armazenar constantes na memória e depois carregá-las
  - Ter registradores que armazenam a mesma constante
    - MIPS possui o registrador `$zero` que armazena 0
  - **Ter instruções especiais que contêm constantes!**

Instruções Imediatas

- MIPS oferece instruções onde uma constante está embutida na própria instrução
- Instruções imediatas contêm 3 operandos:
  - destino, fonte, constante

```
addi a,b,2
```

→

```
a = b + 2
```

- Existe a adição imediata (addi), mas não existe a subtração imediata
  - Subtração : Soma com uma constante negativa

Torne rápido o caso mais comum

Adição Imediata

Código C

```
a = b + 8;

a = a - 2;
```

a em $s1, b em $s2

Código Assembly MIPS

```
addi $s1, $s2, 8
addi $s1, $s1, -2
```

Formato da Instrução `ADDI`

Formato I de Instrução

op	rs	rt	constant
6 bits	5 bits	5 bits	16 bits

op	opcode da instrução
rs	registrador fonte
rt	registrador destino
constant	constante embutida na instrução

Representando ADDI na Máquina

- Número dos registradores
  - **$s0 - $s7: 16 - 23**
  - **$t0 - $t7 : 8-15**
  - **$t8-$t9 : 24-25**

Código Assembly MIPS

```
addi $s1,$s2, 8
```

	$s2	$s1	
8	18	17	8
op	rs	rt	constant